

Problem Set 1

Given on: 13th September 2024
 Due on: 27th September 2024
 Accepted till: 4th October 2024
 Total: 50 points + 4 bonus points

Question 1

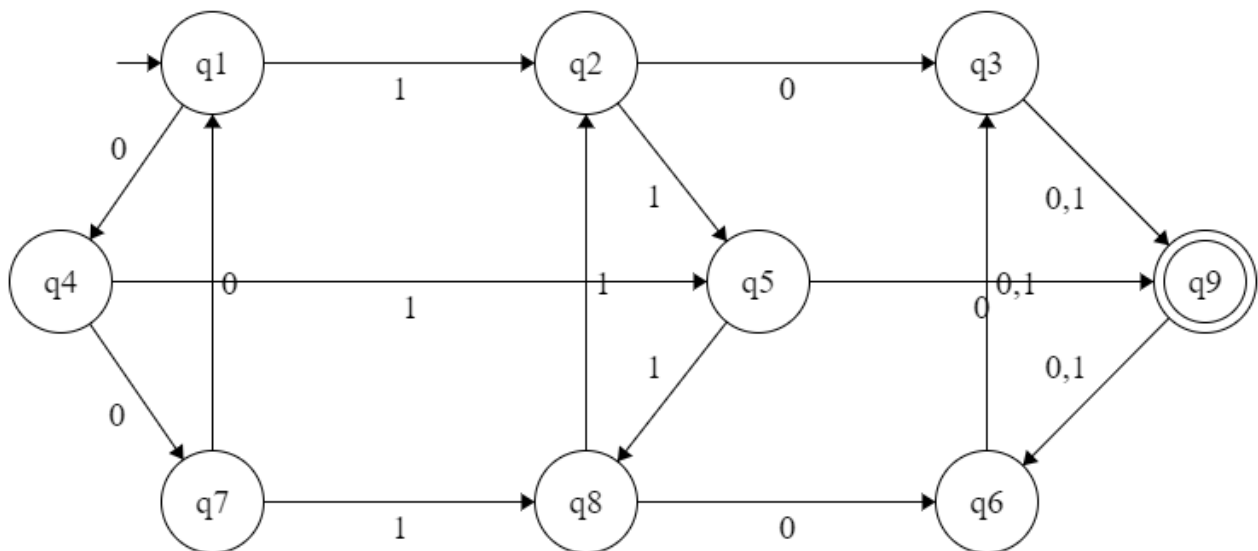
[20 points]

For each of the following languages L over the alphabet $\Sigma = \{0,1\}$, give a regular expression that represents L and describe a DFA that recognizes it.

- (a) All strings of length divisible by 3 such that '10' is a substring.
 (b) All strings that contain '101' as a substring followed by an even number of 1's.

Solution

Figure (1): question 1 (a) DFA solution



Total = 5 marks

- + 1 for attempting DFA
- + 1 for determining substring "10"
- + 1 for performing length check of multiples of 3
- + 1 for attempting to put them together
- + 1 for all correct

Figure (2): question 1 (a) RE solution

$$(\Sigma\Sigma\Sigma)^* \cdot [[\Sigma 10] \cup [10\Sigma] \cup [\Sigma\Sigma 10\Sigma\Sigma]] \cdot (\Sigma\Sigma\Sigma)^*$$

OR

Figure (3): question 1 (a) RE solution

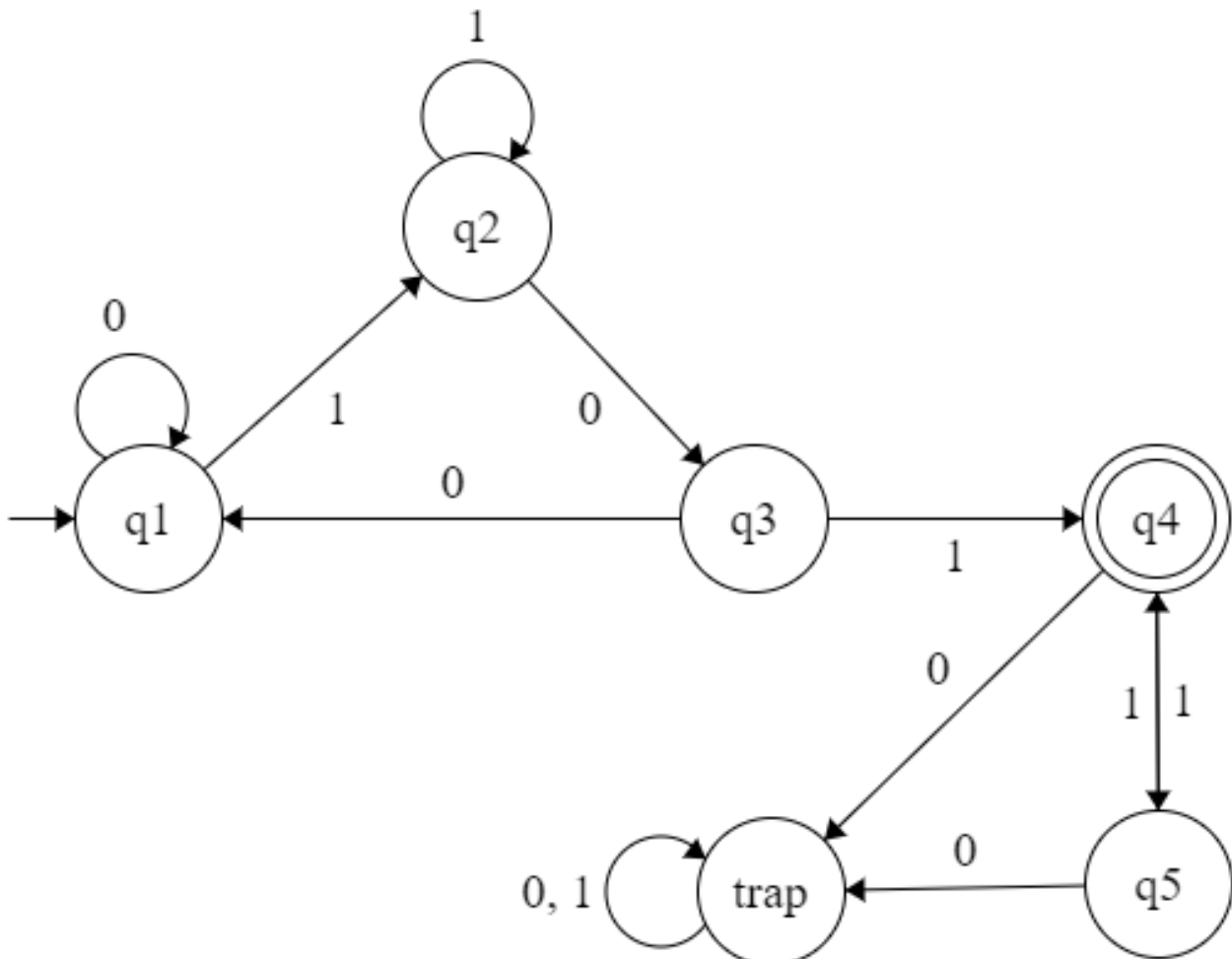
$$((0 \cup 1). (0 \cup 1). (0 \cup 1))^* .$$

$$[[(0 \cup 1).10] \cup [10.(0 \cup 1)] \cup [(0 \cup 1). (0 \cup 1).10. (0 \cup 1). (0 \cup 1)]] .$$

$$((0 \cup 1). (0 \cup 1). (0 \cup 1))^*$$
Total = 5 marks

- + 1 for attempting to find RE
- + 0.5 for sub-string '10' at first position, one Σ after
- + 0.5 for $(\Sigma\Sigma\Sigma)^*$ before AND after the '10' sub-string
- + 0.5 for sub-string '10' at the second position, one Σ before
- + 0.5 for $(\Sigma\Sigma\Sigma)^*$ before AND after the '10' sub-string
- + 0.5 for sub-string '10' at third position, $\Sigma\Sigma$ before and after '10'
- + 0.5 for $(\Sigma\Sigma\Sigma)^*$ before AND after the '10' sub-string
- + 1 for all correct

Figure (4): question 1 (b) DFA solution

**Total = 5 marks**

- + 1 for attempting DFA
- + 1 for determining sub-string "101"
- + 1 for performing an even check on 1's after the sub-string "101"
- + 1 for rejecting 0's after sub-string "101"
- + 1 for fully correct and working FA

Figure (5): question 1 (b) RE solution

$$(0 \cup 1)^* 101 (11)^*$$

OR

Figure (6): question 1 (b) RE solution

$$\Sigma^* 101 (11)^*$$
Total = 5 marks

- + 1 for attempting to make RE
- + 1 for "101" sub-string in RE
- + 1 for Σ^* before the "101" sub-string
- + 1 for even length count of 1's after "101" sub-string
- + 1 for correct RE

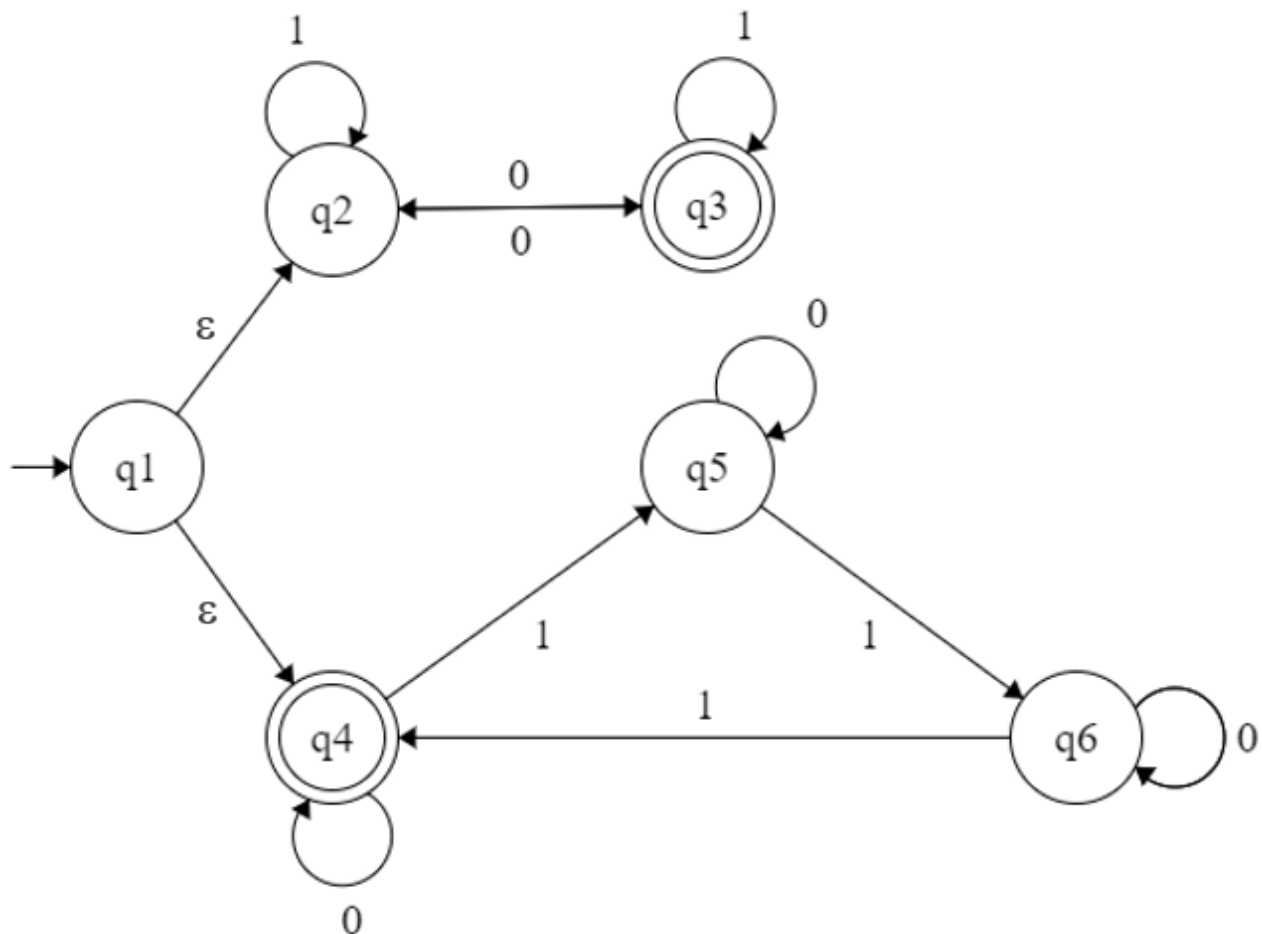
Question 2**[10 points]**

For each of the following languages L over the alphabet $\Sigma = \{0, 1\}$, construct an NFA with the specified number of states recognizing L .

- (a) All strings have an odd number of 0's or 1's in multiples of 3, in six states.
 (b) All strings are in the form $1^*0^*1^+$ with three states.

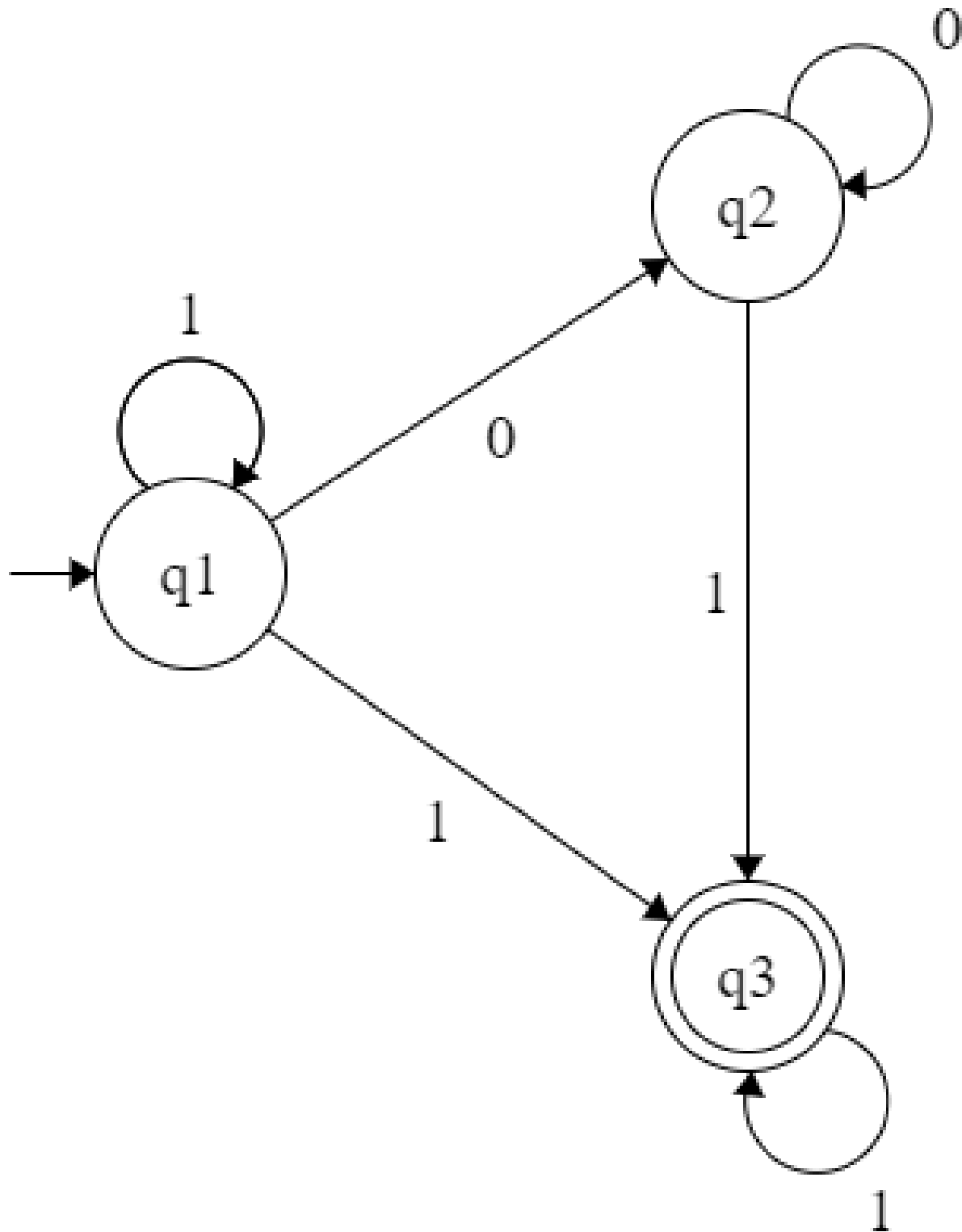
Solution

Figure (7): question 2 (a) NFA solution

**Total = 5 marks**

- + 1 for attempting to make NFA not DFA
- + 1 for using exactly 6 states
- + 1 for 1 in multiples of 3
- + 1 for odd 0s
- + 1 for fully correct FA

Figure (8): question 2 (b) NFA solution

**Total = 5 marks**

- + 1 for attempting to make NFA
- + 1 for used exactly 3 states
- + 1 for strings of only 1's should be accepted (ex 1, 11, 1*)
- + 0.5 for rejecting all strings ending in 0, they should end in 1
- + 0.5 for all strings following the 101 pattern
- + 1 for fully correct FA

Question 3**[10 + 4 points]**

Convert the following NFA's to equivalent DFA's. *Bonus:* find a regular expression for them.

Figure (9): question 3 (a)

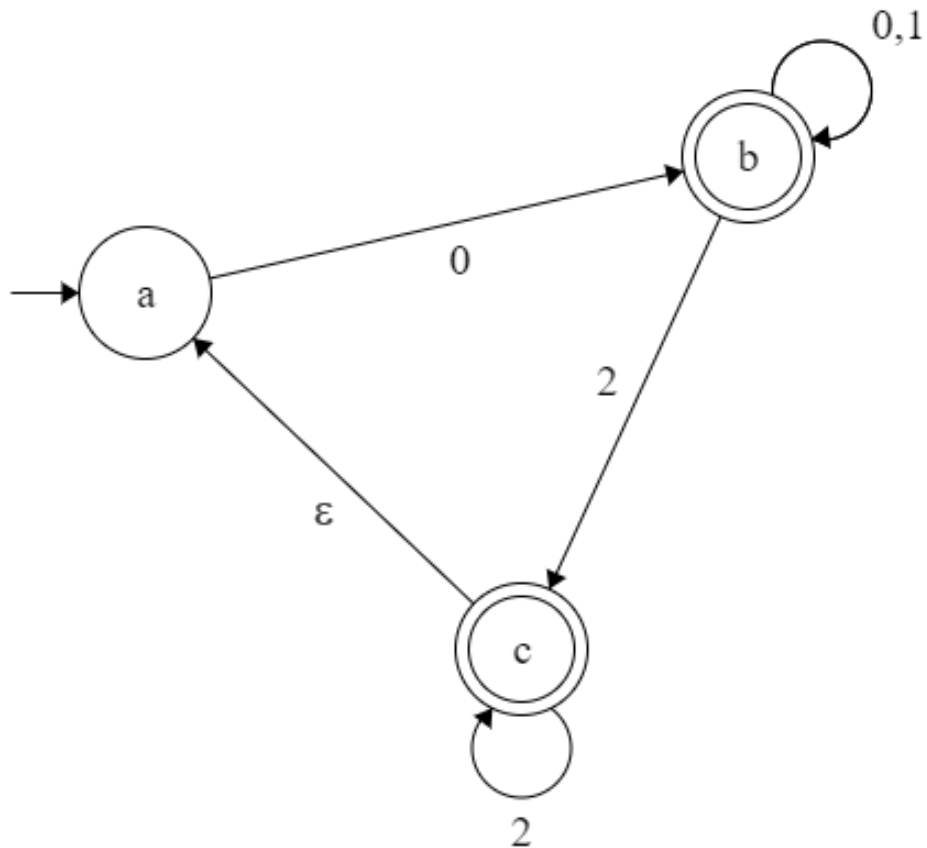
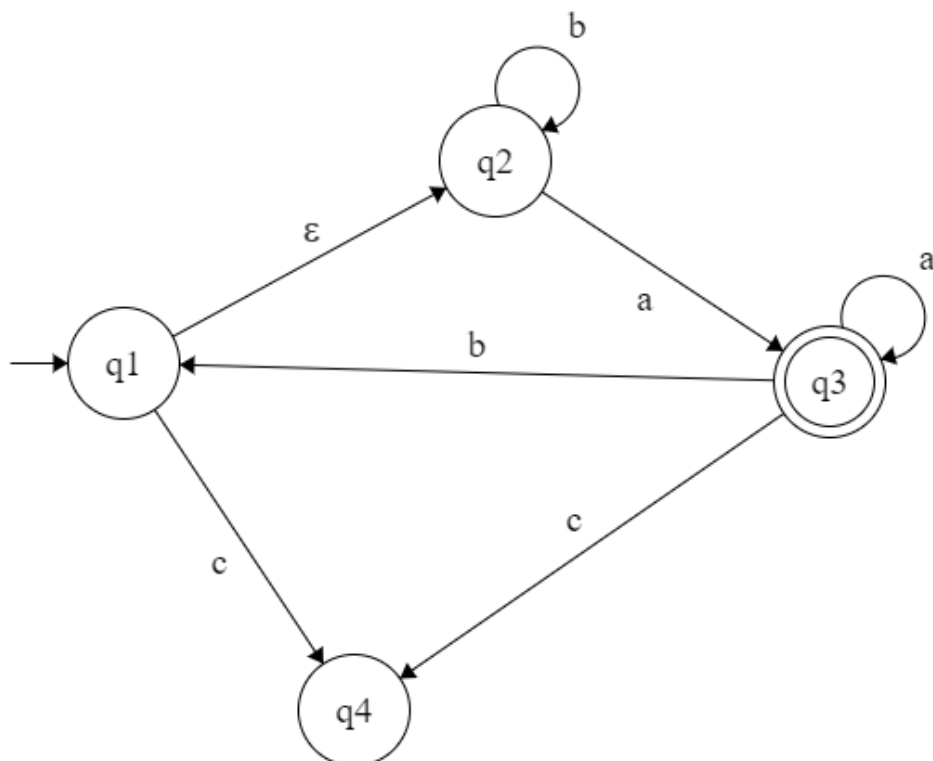


Figure (10): question 3 (b)



Solution

Figure (11): question 3 (a) tabular solution

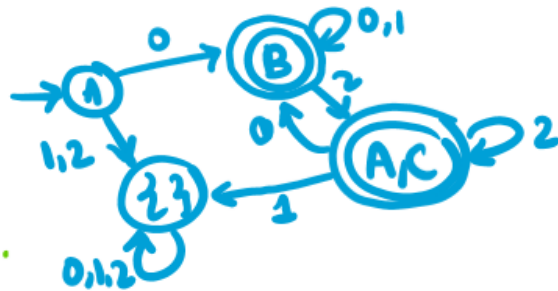
S.R NO.	States	Σ / Characters		
		0	1	2
1	{}	{}	{}	{}
2	A (start state)	B	{}	{}
3	B	B	B	A, C
4	C	{}	{}	A, C
5	A, B	B	B	A, C
6	B, C	B	B	A, C
7	A, C	B	{}	A, C
8	A, B, C	B	B	A, C

Red color states = Accepting states

Highlighted states = required to show states

Accepts all the strings that start with 0

RE = $(0+1*2^*)^+$
 if the string contains a 2 and does not end in 2, it must be followed by a 0.



Total = 5 marks

+ 1 for attempting to make DFA

+ 1 for all states shown

+ 3 for all correct arrows

- 0.5 for each incorrect arrow

Figure (12): question 3 (a) bonus q solution

Accepts all the strings that start with 0

RE = $(0+1*2^*)^+$
 if the string contains a 2 and does not end in 2, it must be followed by a 0.

Total = 2 marks

+ 1 for correct pattern identified

+ 1 for correct regular expression

Figure (13): question 3 (b) tabular solution

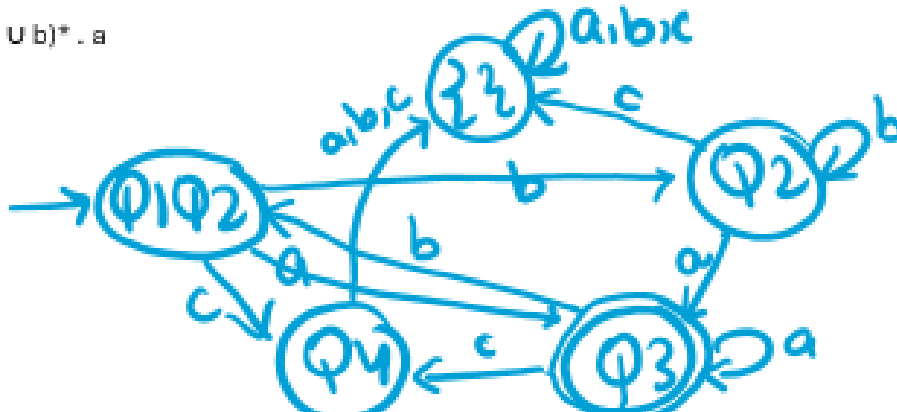
S.R NO.	States	Σ / Characters		
		a	b	c
1	$\{\}$	$\{\}$	$\{\}$	$\{\}$
2	Q1	$\{\}$	$\{\}$	Q4
3	Q2	Q3	Q2	$\{\}$
4	Q3	Q3	Q1, Q2	Q4
5	Q4	$\{\}$	$\{\}$	$\{\}$
6	Q1, Q2 (start state)	Q3	Q2	Q4
7	Q1, Q3	Q3	Q1, Q2	Q4
8	Q1, Q4	$\{\}$	$\{\}$	Q4
9	Q2, Q3	Q3	Q1, Q2	Q4
10	Q2, Q4	Q3	Q2	$\{\}$
11	Q3, Q4	Q3	Q1, Q2	Q4
12	Q1, Q2, Q3	Q3	Q1, Q2	Q4
13	Q1, Q2, Q4	Q3	Q2	Q4
14	Q1, Q3, Q4	Q3	Q1, Q2	Q4
15	Q2, Q3, Q4	Q3	Q1, Q2	Q4
16	Q1, Q2, Q3, Q4	Q3	Q1, Q2	Q4

Red color states = Accepting states

Highlighted states = required to show states

Accepts all the strings that end in an "a" and do not contain a "c"

RE = $(a \cup b)^* \cdot a$



Total = 5 marks

- + 1 for attempting to make DFA
- + 1 for all states shown
- + 3 for all correct arrows
- 0.5 for each incorrect arrow

Figure (14): question 3 (b) bonus q solution

Accepts all the strings that contain an end in an "a" and does not contain a "c"

RE = $(a \cup b)^* \cdot a$

Total = 2 marks

- + 1 for correct pattern identified
- + 1 for correct regular expression

Question 4

[10 points]

The suffix of a language L denoted $SUFFIX(L)$ is a set of all suffixes of strings in L . For example, if $L = \{ab, abc, b\}$, then $SUFFIX(L) = \{ab, b, bc, c, \epsilon\}$.

Show that if L is a regular language, then $SUFFIX(L)$ is also regular by constructing a new automation that accepts all suffixes of L . Outline the process of modifying the DFA to recognize $SUFFIX(L)$.

Solution

To show that if L is a regular language, then $SUFFIX(L)$ is also regular, we can construct a new deterministic finite automaton (DFA) that accepts all suffixes of the strings in L . The process involves transforming the DFA that accepts L into one that accepts $SUFFIX(L)$.

Outline of the Process:

1. **Original DFA for L :** Suppose you have a DFA $M = (Q, \Sigma, \delta, q_0, F)$ that accepts the language L . Here:
 - Q is the set of states,
 - Σ is the input alphabet,
 - δ is the transition function $\delta : Q \times \Sigma \rightarrow Q$,
 - q_0 is the start state,
 - F is the set of accepting states.
2. **Suffixes and Intuition:** The key observation is that any suffix of a string in L is a portion of the string that begins from some point onward in the string. Therefore, the DFA needs to accept all substrings of a string that the original DFA accepts, starting from any state reachable from the start state q_0 .

3. **Modified DFA:** The new DFA for $\text{SUFFIX}(L)$ can be constructed by modifying the original DFA M . The idea is to make the DFA accept from any state it can reach, not just from the start state.

- **Start from any state:** We want the new DFA to be able to accept suffixes that could start from any point in the original DFA's processing of a string. To do this, we can make all states of M (or the reachable ones) potential starting states in the new DFA.
 - Create a new DFA $M' = (Q', \Sigma, \delta', q'_0, F')$, where:
 - $Q' = Q \cup \{q'_0\}$ (add a new start state q'_0),
 - q'_0 is a new start state that transitions to all states in Q ,
 - $F' = F$ (keep the same set of accepting states),
 - For every $q \in Q$ and every $a \in \Sigma$, $\delta'(q'_0, a) = \delta(q_0, a)$ (define transitions from q'_0 to simulate starting from anywhere).
- **Transition structure:** The DFA M' will start from q'_0 , which is a "super start" state that has ϵ -transitions (empty string transitions) or immediate transitions to all the original states in Q . This means that from q'_0 , the DFA can effectively start simulating any part of the computation of the original DFA M from any state, enabling it to accept suffixes of strings in L .

4. **Acceptance from intermediate states:** Once the DFA can start from any state, it will naturally accept suffixes. This is because, by construction, it will traverse any valid path from any reachable state, eventually reaching an accepting state.

Example:

Suppose the original DFA M accepts the language $L = \{ab, abc, b\}$. The new DFA M' will accept suffixes such as ab, b, bc, c, ϵ .

Conclusion:

Since DFAs are closed under modifications that allow changing the start state and redefining transitions, the modified DFA M' will still be a DFA. Therefore, $\text{SUFFIX}(L)$ is a regular language if L is regular. The construction outlined ensures that the new DFA accepts all suffixes of strings in L .

Total = 10 marks

- + 1 for attempting the question
- + 1 Proper presentation of proof in paragraphs
- + 1 Did not provide an example DFA or use the example given in question
- + 1 for defining a generalized DFA
- + 1 for creating a new DFA with the same states and one extra state
- + 1 for creating a new DFA with the new extra state as the start state
- + 1 for creating a new DFA with the same accepting states

- + 1 for creating a new DFA with the same transitions + new transitions of that new state to ϵ -transition to every state in the DFA
- + 1 for understanding how now the new DFA can start from any state and therefore accept all suffixes of the language
- + 1 for fully correct proof (only given when the proof is fully correct)